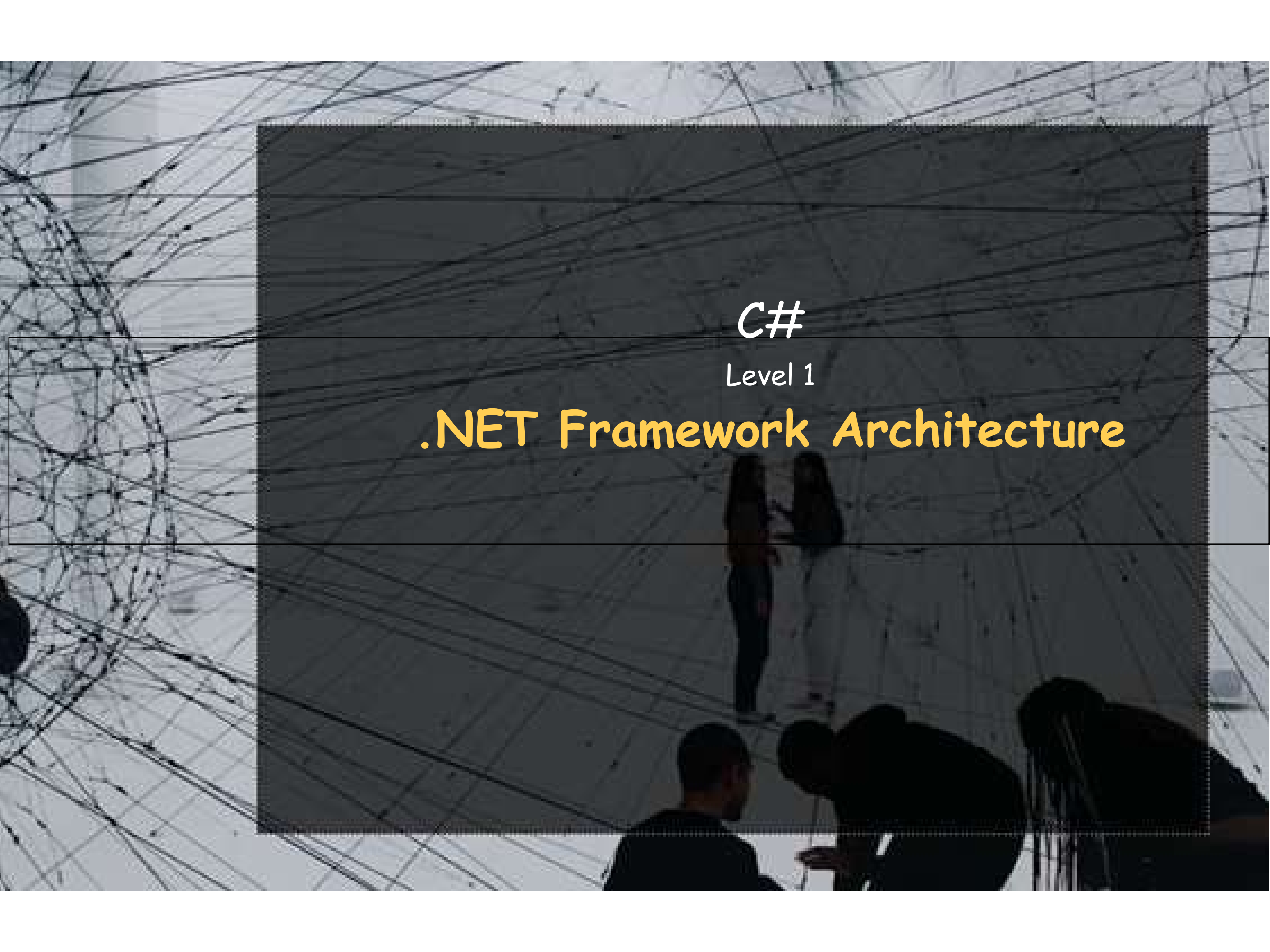


The background is a blurred image of a computer monitor. It shows a dark-themed code editor with various lines of code in different colors (blue, green, yellow). At the bottom, a Windows taskbar is visible with several application icons including a web browser, file explorer, and various development tools.

C#

Level 1

.NET Framework Architecture



C#

Level 1

.NET Framework Architecture

🔍 Definition

■ .NET Framework Architecture

▨ The .NET Framework Architecture describes how different components work together to build and execute applications.

▨ Major concepts include languages, application models, class libraries, and the Common Language Runtime.

Languages



Application Models



.NET Framework Class Library



CLR



Operating System



Hardware

Explanation

Languages

-  .NET Framework supports languages such as *C#*, Visual Basic, *F#*, and other .NET languages.

-  Language compilers convert source code into Intermediate Language.

Application Models

-  Windows Forms → Windows desktop applications

-  Web Forms → ASP.NET web applications

-  Console Applications → command-line applications

.NET Framework Class Library

-  Provides reusable classes, types, and APIs.

-  Supports strings, numbers, dates, collections, files, networking, database access, diagnostics, threading, and reflection.

Base Class Library

-  BCL provides core .NET types and fundamental functionality.

-  Examples include `System.String`, `System.DateTime`, `System.IO`, `System.Threading`, and `System.Collections`.

Common Language Runtime

-  CLR is the execution engine of the .NET Framework.

-  It manages code execution, memory, garbage collection, exceptions, type safety, and threads.

Procedures

1. Write the application using a .NET-supported language.
2. Use an application model appropriate for the application type.
3. Use the .NET Framework Class Library and BCL APIs.
4. Compile the source code into IL.
5. Run the application through the CLR.
6. Let the CLR provide runtime services such as garbage collection and memory management.

💡 Example

Suppose you create a *C#* application that reads a file:
C# Application



System.IO



.NET Framework Class Library



CLR



Windows



File System

- ▣ *C#* → language used to write the application
- ▣ *System.IO* → API used to work with files
- ▣ *CLR* → executes and manages the application
- ▣ *Windows* → underlying operating system

Note

- ◆ .NET Framework is primarily a Windows-focused, legacy implementation of .NET.
- ◆ The current unified platform is simply called .NET.
- ◆ .NET 6 was an LTS release, but it is not the latest .NET version.

⊘ Warning

◆ Do not confuse:

▨ CLR → Runtime/execution engine

▨ BCL/FCL → Libraries and APIs

▨ C# → Programming language

▨ .NET Framework → Older Windows-focused .NET implementation



Comparison



CLR

- ✓ Executes .NET applications
- ✓ Provides runtime services
- ✓ Manages memory and garbage collection
- ✓ Handles exceptions and type safety



Class Library

- ✓ Provides reusable APIs
- ✓ Handles common programming tasks
- ✓ Includes I/O, collections, networking, and other functionality
- ✓ Used by applications running on the CLR

✦✦ Summary

- ✓ Languages such as *C#* are used to write applications
- ✓ Application Models provide technologies for different application types
- ✓ Class Library provides reusable APIs and functionality
- ✓ BCL provides core .NET types and services
- ✓ CLR is the execution engine
- ✓ Garbage Collector manages managed memory automatically
- ✓ ADO.NET provides database access APIs
- ✓ I/O provides file and data input/output functionality
- ✓ Reflection allows runtime inspection of types and metadata
- ✓ Threading provides concurrency-related functionality
- ✓ .NET Framework is primarily Windows-focused and legacy
- ✓ Modern .NET is the unified, cross-platform successor to .NET Framework

Thank you 😊